PEKOM

UNIVERSITI MALAYA

UMHackathon 2026

PRODUCT REQUIREMENT DOCUMENT

(PRD)

UMHackathon 2026

umhackathon@um.edu.my

Team Name: hipp0senpai

Domain 1: AI Systems & Agentic Workflow Automation

Project Title: FutureJob

Team Member
Koo Wei Gang
New Wei Jun
Tan Choon Da

Table of Content

1. Project Overview 4
2. Background & Business Objective 4
3. Product Purpose 5
    3.1. Main Goal of the System 5
    3.2. Intended Users (Target Audience) 5
4. System Functionalities 5
    4.1. Description 5
    4.2. Key Functionalities 5
    4.3. AI Model & Prompt Design 6
    4.3.1. Model Selection: 6
    4.3.2. Prompting Strategy: 6
    4.3.3. Context & Input Handling: 6
    4.3.4. Fallback & Failure Behavior: 6
5. User Stories & Use Cases 6
6. Features Included (Scope Definition) 7
7. Features Not Included (Scope Control) 7
8. Assumptions & Constraints 7
9. Risks & Questions Throughout Development 8

Project Name: FutureJob

Version: 2.0

1. Project Overview

- **Problem Statement:**

  The modern job search ecosystem is fundamentally broken for both applicants and employers, primarily due to the limitations of legacy Applicant Tracking Systems (ATS). Across Malaysia, the employment sector relies on fragmented, manual, and unstructured workflows. Candidates submit varied, unstructured resumes, and recruiters must manually interpret this data against rigid job requirements. Based on our researches, we found that the legacy job search ecosystem have the following pain points:

  - **The "Resume Black Hole" & Friction:** Candidates spend hours manually re-entering data into fragmented forms, only to face a "black hole" of silence, with no actionable feedback upon rejection.
  - **Rigid Keyword Gatekeeping:** Current systems rely on Boolean keyword matching. A candidate highly skilled in "Next.js" might be automatically filtered out if the legacy ATS strictly requires the keyword "React," lacking the semantic intelligence to understand the technical correlation.
  - **The False Positive Avalanche:** Conversely, employers are flooded with low-quality applications from candidates who simply stuff their resumes with keywords, creating a high noise-to-signal ratio that slows down recruitment workflows.

- **Target Domain:** AI Systems & Agentic Workflow Automation (HR Tech / Recruitment)
- **Proposed Solution Summary:**

  FutureJob serves as a stateful, adaptive workflow engine. It utilizes an AI General Language Model (GLM) as the central reasoning orchestrator to transform unstructured candidate data and conversational job descriptions into a structured, bidirectional matchmaking workflow. It automates profile generation, intelligent applicant filtering, match scoring, and personalized feedback generation.

2. Background & Business Objective

- **Background of the Problem:** Traditional recruitment platforms act as static databases requiring extensive manual data entry and human interpretation. They lack semantic intelligence, causing high application abandonment rates and forcing recruiters to manually screen hundreds of incompatible resumes.
- **Importance of Solving This Issue:** By accelerating the pre-screening process through semantic AI matchmaking, companies can drastically reduce their time-to-hire. For job seekers, breaking the "black hole" cycle by providing automated, personalized improvement feedback transforms the job hunt from a frustrating chore into a guided career development path.
- **Strategic Fit / Impact:** FutureJob perfectly aligns with the domain of Agentic Workflow Automation. The GLM does not function as a simple chatbot; it acts as a stateful engine that actively orchestrates system transitions, handles ambiguous data proactively, and makes autonomous routing decisions.

3. Product Purpose

3.1. Main Goal of the System

To build an autonomous, AI-driven workflow engine that intelligently bridges the gap between unstructured talent profiles and strict corporate job requirements, facilitating end-to-end automation from resume parsing to interview scheduling.

3.2. Intended Users (Target Audience)

- Students / Job Seekers
- Corporate Recruiters / HR Managers
- System Administrators (for verification)

4. System Functionalities

4.1. Description

The system operates as a high-speed matchmaking and orchestration engine. Using a Next.js and Firebase (Firestore) stack, the platform leverages the GLM to parse unstructured inputs into strict JSON entities, calculate bidirectional fit, and autonomously execute application routing and feedback delivery.

4.2. Key Functionalities

- **Unstructured-to-Structured Parsing:** Translates messy, human-language notes into standardized, high-fidelity JSON professional profiles and job requirements.
- **Agentic Matchmaking & Scoring:** Contextually compares candidate skills against job needs to generate a match percentage and a human-readable reasoning trace.
- **Autonomous Workflows:** Automatically dispatches applications for matches exceeding an 85% threshold and generates multi-step "Skill Quest" feedback roadmaps upon candidate rejection.
- **Integrated Communication:** Facilitates continuous connection through an embedded chat proxy and interview session manager once a candidate is approved.

4.3. AI Model & Prompt Design

4.3.1. Model Selection

We selected the Z.AI GLM (Gemini API) due to its superior instruction-following capabilities for enforcing strict JSON schemas, and its large context window, which is required for multi-step reasoning when comparing complex job requirements against detailed candidate profiles.

4.3.2. Prompting Strategy

We utilize Agentic, Multi-Step Prompting with strict JSON Schema enforcement. For profile generation, we use zero-shot entity extraction. For the rejection feedback loop, we use context-heavy prompting (feeding the GLM the job schema, candidate schema, and company rejection reason simultaneously) to synthesize highly specific, actionable advice.

4.3.3. Context & Input Handling

To prevent context window overflow and excessive token billing, the frontend limits text area inputs to 5,000 characters. In the Next.js API route, the system slices any incoming string to a strict character limit before injecting it into the prompt payload. If a user attempts to bypass limits via API directly, the system truncates the excess text, ensuring the core parsing task focuses only on the most relevant beginning and middle segments of the unstructured data.

4.3.4. Fallback & Failure Behavior

To prevent hallucinations, the API calls enforce a low temperature (e.g., 0.1) for deterministic output. If the GLM returns malformed JSON or hallucinates keys outside the TypeScript schema, the Next.js API utilizes a built-in try/catch block that triggers up to 3 silent retries. If it still fails, the API returns a graceful HTTP 500 error state, and the frontend prompts the user with a fallback option to input their details manually rather than leaving them blocked.

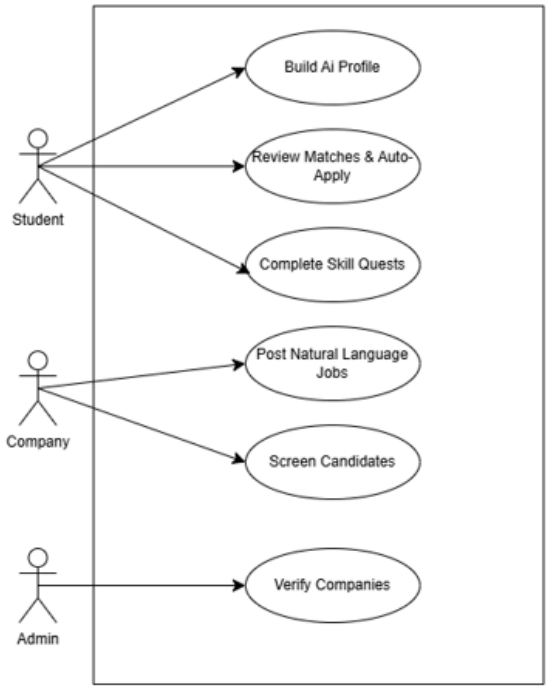
5. User Stories & Use Cases

- **User Stories:**
  - *"As a Student, I want to paste my messy LinkedIn summary so that the system automatically builds a polished, structured profile for me without manual entry."*
  - *"As a Recruiter, I want the system to show me only candidates who have an AI match score of >85% so that I don't waste time reading irrelevant resumes."*
  - *"As a Rejected Candidate, I want the system to tell me exactly what skills I am missing (Skill Quests) so that I can improve and reapply."*

- **Use Cases (Main Interactions):**

  - **Profile Ingestion Flow:** Student pastes raw text. The Next.js BFF sends it to the GLM with a schema. The GLM returns strict JSON. System saves to Firestore.
  - **Agentic Match & Apply Flow:** System periodically compares student profile against open jobs. GLM returns match scores. If >85%, an Application document is auto-created.
  - **Feedback Loop Flow:** Recruiter hits "Reject". System sends rejection context to GLM. GLM generates a "Skill Quest" and saves it to the student's dashboard.

- **Use Case Diagram:**



6. Features Included (Scope Definition)

- AI Profile Builder (Unstructured text to structured JSON).
- Natural Language Job Posting (Conversational to exact requirements).
- Bidirectional Match Scoring Engine with Reasoning Traces.
- Automated Application Generation (Score > 85%).
- Multi-step "Skill Quest" feedback loops for rejected applicants.
- Admin Verification Dashboard (Security Gateway).

7. Features Not Included (Scope Control)

- Native Video/WebRTC Interviewing functionality.
- 3D Gamified virtual workspaces for quest completion.
- Payroll, ERP, and deeper internal HR software integrations.
- Complex resume PDF generation with custom styling engines

8. Assumptions & Constraints

- **LLM Cost Constraint:** Heavy bidirectional matching can quickly consume token budgets.
- **Design Decision:** We implement server-side caching mechanisms to store the Match Score between specific studentId and jobId pairs. The GLM is only invoked if the student's core profile has been updated since the last match calculation.
- **Technical Constraints:** Firestore (NoSQL) lacks complex joining capabilities. Therefore, dashboards must rely on highly denormalized data (e.g., embedding company names and match scores directly into Application documents) to maintain fast read speeds.
- **Performance Constraints:** Serverless execution timeouts on Vercel (typically 10-15s on free tiers). Multi-step AI reasoning must be prompt-optimized to return quickly to prevent UI hanging.

9. Risks & Questions Throughout Development

- **AI Hallucination & Drift:** How do we ensure the GLM doesn't fabricate skills that the candidate never mentioned in their unstructured text? (Addressed via strict temperature: 0.1 and explicit zero-shot system prompts).
- **Bias in Matching:** Will the AI unfairly score candidates based on formatting rather than substance? (Addressed by parsing everything into a uniform JSON schema first, and only comparing schemas).
- **Index Building Delays:** Firebase requires composite indexes that take minutes to build. How do we prevent blank dashboards during hackathon deployments? (Addressed via client-side memory sorting fallbacks).